Design layer

the whole layer, as one document

Contents

1 Retrace 4

1.1 Foundation 4
1.2 System and ownership 4
1.3 Recording and replay 5
1.4 Client delivery 5
1.5 Ingestion and archival 6
1.6 Example application and viewer 7

Glossary 7

ROOT DESIGN

Retrace

1 Retrace

1.1 Foundation

Retrace records application messages, delivers durable sessions and reconstructs them for inspection.

GOAL

Follow a session across capture, storage and replay

A developer can reproduce an application state from a web or Tauri session, search recordings by user reference, and replay a recording after the collector restarts.

GOAL

Compose small runtime owners

The portable recording core has no Lustre dependency. A client owns a local delivery queue. The server coordinates PostgreSQL and S3. The example viewer optionally composes Time Travel.

NO-GOAL

Universal screen recording

The initial release reconstructs explicitly recorded application state. It does not capture pixels, operating-system dialogs, arbitrary DOM mutations or native screens.

INVARIANT

Playback does not execute application effects

The viewer uses an application-supplied pure reducer and inert view. HTTP, native calls and other effects remain in the live application. Function types alone cannot prove purity.

convention

INVARIANT

Acknowledgement follows durable commit

A Acknowledgement means PostgreSQL committed the immutable Batch. S3 failure cannot erase that accepted copy.

partial

PRINCIPLE

The application owns safe meaning

Applications select replay-safe messages, state projections, codecs, user references and Replay contract identifiers before capture. Formatters do not establish redaction.

composition

This initial system is a local development example with explicit runtime owners. Deployment authentication, tenant isolation, retention and erasure require separate policy before a hosted deployment. ADR-0001

1.2 System and ownership

The application, client queue, collector and viewer cooperate through explicit values and durable receipts.

Unit	Owns	Caller owns
Portable package	Recording cursor, reconstruction, envelope validation	Types, codecs, clock, deterministic reducer
Client SDK	Bounded durable queue, serialized delivery, retries	Capture policy, endpoint, identity, storage selection

Unit	Owns	Caller owns
Server package	Atomic admission, session index, archival and retrieval	Database pool, object-store configuration, worker lifetime
Example applications	Mist routes, Lustre interface, Tauri shell and viewer	Starting and stopping local resources

- A `Session` has an immutable ID, user reference, build, platform and replay contract. Multiple devices or tabs use distinct IDs. User reference is searchable metadata, not authorization.
- A recording cursor owns sequence and elapsed-time monotonicity. A `Entry` advances it once, including a message that changes no state. A `Checkpoint` starts reconstruction at its declared cursor.
- Transport format version and application replay compatibility are separate. The initial transport uses bounded JSON strings for application payloads so the collector need not import application types.
- The example uses PostgreSQL 16, a caller-configured S3-compatible service, Mist on BEAM, and a shared Lustre frontend for browser and Tauri. Nix supplies the development tools.

1.3 Recording and replay

Pure values retain application types and report invalid history without advancing a player.

BEHAVIOR

Invalid history preserves the replay cursor

Given a player at a validated checkpoint

When an entry has a gap, duplicate position, different session or contract, backwards time, or a failing application reducer

Then the operation returns the corresponding typed failure and the previous player remains usable

level: boundary area: Reconstruction

BEHAVIOR

A checkpoint can restart reconstruction

Given a compatible checkpoint and subsequent contiguous entries

When the caller reconstructs the recorded range

Then the final state and absolute sequence can be inspected and compared with a supplied checkpoint

level: boundary area: Reconstruction

- Start validates identity. Record increments sequence and returns an immutable entry. Resume validates a supplied cursor; the caller must first recover its matching outbox and state. All sequence and elapsed-time integers fit the JavaScript exact-integer range.
- A batch contains at most 128 entries and at most one MiB of encoded bytes. Its checkpoint precedes its entries; all origins match its session. A batch is nonempty. JSON decoding does not prove the application's model or event validity.
- Compatibility checks do not prove reducer equivalence or authenticate a checkpoint. The application owns those claims.

1.4 Client delivery

The client persists before uploading and removes a batch only after its matching durable receipt.

BEHAVIOR

Interrupted delivery is recoverable

Given a batch confirmed as saved locally

When the network fails or the client runtime restarts before acknowledgement

Then the batch remains available for retry with the same session and sequence

level: boundary area: Delivery

BEHAVIOR

Capture failure stays visible

Given the local queue is full or its storage write fails

When the application attempts to record another transition

Then capture reports its failure without claiming that transition was saved

level: interface area: Capture

- A client runtime has one writer and one sequential upload loop. Each page instance starts a fresh session; it still drains durable batches left by earlier instances. Upload duplicates caused by overlapping runtimes are safe on the server.
- Browser persistence uses IndexedDB. Tauri persistence uses its native store plugin. The SDK returns explicit persistence and delivery status through the application's callback. It owns retry timers and exposes shutdown.
- The initial SDK caps its queue by item count and bytes. A live transition and asynchronous local capture are separate commits; an abrupt termination before save confirmation can lose that transition. Persisted batches survive a normal runtime restart; device failure, storage eviction and mobile background execution are outside this guarantee.

1.5 Ingestion and archival

The server serializes admission per session and moves accepted recording bytes to object storage without a loss window.

BEHAVIOR

Duplicates are distinguished from conflicts

Given a batch was durably accepted

When a caller retries its range

Then identical content returns the same acknowledgement and different content is refused

level: boundary area: Admission

BEHAVIOR

Session admission is ordered and atomic

Given a session's durable cursor

When a new batch is admitted

Then its checkpoint must match that cursor and its fixed session metadata must agree before the batch and index advance together

level: boundary area: Admission

BEHAVIOR

Archival failure preserves accepted data

Given a committed batch still held in the database

When object upload fails

Then the database copy remains readable and eligible for a later retry

level: boundary area: Archival

BEHAVIOR

Archived sessions remain replayable

Given a batch was successfully archived

When the collector restarts and the viewer requests that session

Then the session remains searchable and its recording is fetched and checked before playback

level: boundary area: Retrieval

- Database states are pending, with payload and digest, and archived, with object key and digest. Upload precedes committing the archived state. A crash before that commit can leave an orphan object; the deterministic key makes retry safe. Failed upload never clears the database payload.
- Admission locks the session row. The initial checkpoint starts at sequence zero. A previously accepted batch is identified by session and starting sequence; its digest permits exact retry checks after its database payload has been removed. Different chunking is a conflict, not automatic deduplication.
- Session search is bounded and supports a user reference. Range retrieval is ordered and bounded. The object adapter returns compressed bytes; reconstruction verifies the stored content digest after decompression.
- The server package borrows a database pool and object-store functions. The example owns the pool and periodic archive worker. Storage functions have explicit read/write failures; no package starts hidden background workers.

1.6 Example application and viewer

A small membership app demonstrates the same session contract in the browser and Tauri.

BEHAVIOR

Inspection leaves live capture running

Given a developer is viewing an earlier replay state

When a live application effect completes

Then the live application records the result while the selected replay remains unchanged

level: interface area: Replay

BEHAVIOR

Replay is inert

Given a compatible stored recording

When the viewer navigates its entries

Then the application state is reconstructed without making native requests or recording new live events

level: interface area: Replay

- The frontend separates live application state, capture state and viewer state. It exposes session/user identity, queued versus acknowledged progress, session search, selection and playback.
- A native platform-information request demonstrates a genuine Tauri command and records its result as an application message. The browser has its own labelled result. Native operating-system UI is represented by events, not a pixel recording.
- The viewer accepts only its supported replay contract and build. It loads stored ranges, decodes application payloads, reconstructs with the pure reducer and feeds inert snapshots to the Time Travel inspector.
- The local collector binds to loopback by default. Local example routes are unauthenticated and must not be treated as a hosted service configuration.

Glossary

Every term the layer declares, with the context that owns it.

Acknowledgement

owned by root

The server's confirmation that a batch is durably committed; it permits the client to remove that batch from its queue.

Batch

owned by root

An immutable checkpoint followed by a bounded contiguous range of entries, used as the unit of upload, deduplication and archival.

Checkpoint

owned by root

A replay-safe application model at a known session sequence and elapsed time.

Entry

owned by root

One accepted, safe application message with a sequence number and monotonic elapsed time inside a session.

Replay contract

owned by root

The application-owned compatibility identifier for its event schema, checkpoint schema and reducer semantics.

Session

owned by root

One application's ordered recording, identified independently from its user's reference and fixed to an application build and replay contract.